Lab 3 — ETL and Data Validation

Track A (tabular fraud-detection) · Week 3 · DS5619 Machine Learning Systems Operations

Setup

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python generate_for_student.py --student-id <your roll number or institute email>
```

This overwrites `data/raw_transactions.csv` with 600 rows containing the same 7 violation TYPES as everyone else's, but at DIFFERENT row positions, seeded from your student ID. Two students never get the same broken rows. Re-running with your own ID always reproduces the same file.

Record your `--student-id` value in `NOTES.md`. The grader re-runs `generate_for_student.py` with the ID you recorded and diffs the result against what you committed under `data/`.

Files

- `src/expectations.py` — implement the four `# TODO` expectation functions.
- `src/etl.py` — implement `run_etl()`. `build_expectation_suite()` and `extract()` are already done for you.
- `data/raw_transactions.csv` — your 600-row dirty input, generated above (don't hand-edit it).
- `config.yaml` — points at the input/output paths; no changes needed.

Background

`data/raw_transactions.csv` is 600 transactions. It is not clean. Seven rows have real, specific problems: some have a null amount, one has a negative amount (a transaction can't cost -$450), one has a merchant category outside the set the business recognizes, one has a country code that isn't real, one has a null card ID, and two rows share a transaction ID that should be unique. **You don't know exactly which rows yet — your validation suite is what's supposed to find them**, and — since your data is personalized — neither does anyone whose answer you might otherwise be tempted to copy.

This is the point of a data contract: the pipeline should refuse to advance rows that violate it, and it should tell you specifically what broke and why, not just "something's wrong."

Your task

Part 1 — `src/expectations.py` (the validation suite, ~40 min)

Implement the four expectation functions marked `# TODO`. Each takes the list of row-dicts and returns a list of `Violation` objects (already defined for you — don't change that class). An empty list means that expectation passed.

- `expect_column_not_null(rows, column)` — no row may have `None`/empty string in `column`.

- `expect_column_positive(rows, column)` — every value in `column` must be a positive number.
- `expect_column_in_set(rows, column, allowed_values)` — every value in `column` must be one of `allowed_values`.
- `expect_column_unique(rows, column)` — no two rows may share the same value in `column`.

Part 2 — `src/etl.py` (the pipeline, ~35 min)

Implement `run_etl(config)`, marked `# TODO`. It must:

1. **Extract**: load `config["input_path"]` (CSV).
2. **Validate**: run the expectation suite defined in `build_expectation_suite()` (already provided — look at it, it tells you exactly which checks apply to which columns) against the extracted rows. Collect every violation from every check — don't stop at the first one.
3. **Transform**: split rows into two groups — rows with zero violations pass through to a `clean_transactions.csv`; rows with any violation go to a `quarantined_transactions.csv`, unmodified, so nothing is silently dropped.
4. **Load**: write both output files, plus a `validation_report.json` summarizing, per expectation, how many violations it found and on which row indices.

```
python src/etl.py --config config.yaml
```

This writes `data/clean_transactions.csv`, `data/quarantined_transactions.csv`, and `data/validation_report.json`. The copies currently in `data/` are placeholders — they'll be overwritten with real content the first time you run a working `run_etl()`.

Self-check

```
pytest tests/ -q
```

This is a self-check, not the grader — passing it means your expectation functions and pipeline wiring work on the known cases, not that you're done.

Deliverables (what you commit)

- `src/expectations.py` and `src/etl.py`, completed.
- The three output artifacts: `data/clean_transactions.csv`, `data/quarantined_transactions.csv`, `data/validation_report.json`.
- A short `NOTES.md`: the `--student-id` value you used (required — see above), plus how many rows ended up quarantined, and does that match the 7 known injected problems? (It won't match exactly — some rows may trip more than one expectation. Explain the discrepancy if there is one.)

Grading checklist

- ☐ `data/raw_transactions.csv` matches what `generate_for_student.py --student-id <NOTES.md value>` actually produces.

- [ ] All four expectation functions correctly identify violations (checked against a held-out set of rows with known problems, not just the ones you can see).
- [ ] `run_etl` quarantines every row with at least one violation, and passes through every row with zero violations — no row is silently dropped or silently kept.
- [ ] `validation_report.json` correctly attributes each violation to the right expectation and row.
- [ ] `NOTES.md` shows you actually looked at what got quarantined and reasoned about it.
- [ ] Meaningful commit history and a working README.

Submission

```
git add -A
git commit -m "Week 3: ETL + validation suite"
git tag week03-submit
git push origin main --tags
```